

Documentación API MangaReader

Equipo de Desarrollo

16 de junio de 2026

Control de Documentos (ISO 9001)

Versión	Fecha	Autor	Descripción
1.0	16 de junio de 2026	Equipo Dev	Versión Inicial del Sistema (Django)
2.0	16 de junio de 2026	Equipo Dev	Migración a FastAPI, Vue Composition API y Cloudfl

Índice

1. Arquitectura del Sistema	3
1.1. Visión General	3
1.2. Tecnologías Principales	3
1.3. Diagrama de Componentes	3
1.4. Flujo de Datos	3
2. Base de Datos y Modelos	4
2.1. Esquema Relacional	4
2.1.1. Modelo de Usuarios (User)	4
2.1.2. Modelo de Mangas (Manga)	4
2.1.3. Modelo de Estructura	4
2.2. Políticas de Integridad	4
3. Protocolos de Seguridad	5
3.1. Control de Acceso (RBAC)	5
3.2. Autenticación	5
3.3. Protección de Infraestructura	5
3.3.1. Rate Limiting	5
3.3.2. Protección de CDN (Hotlink Protection)	5
3.4. CORS	5
4. Manual de Operaciones y Despliegue	6
4.1. Ciclo de Vida del Software (SDLC)	6
4.2. Monitorización y Alertas	6
4.3. Procedimientos de Recuperación (Disaster Recovery)	6
4.3.1. Restauración de Servicio	6
4.3.2. Backup de Datos	6

5. Referencia API	7
5.1. Autenticación	7
5.2. Mangas	7
5.3. Monitorización	7

1. Arquitectura del Sistema

1.1. Visión General

MangaReader es una plataforma de distribución de contenido digital (manga/manhwa) construida sobre una arquitectura "Ponytail" ligera y modular con FastAPI. El sistema está diseñado para alta disponibilidad, baja latencia, escalabilidad horizontal y seguridad robusta.

1.2. Tecnologías Principales

- **Backend:** Python 3.13 + FastAPI.
- **ORM y Validación:** SQLAlchemy, Alembic y Pydantic.
- **Base de Datos:** MySQL (Producción), SQLite (Desarrollo).
- **Infraestructura:** Cloudflare Workers (Protección B2/Hotlink) y Railway (PaaS).
- **Frontend:** Vue.js 3 (Composition API) optimizado sin gestores de estado complejos (sin Pinia).

1.3. Diagrama de Componentes

El sistema se divide en los siguientes módulos lógicos:

1. **Routers / Endpoints:** Puntos de entrada HTTP manejados por FastAPI (ej. `catalog`, `reader`).
2. **Servicios (Services):** Lógica de negocio encapsulada y separada de las rutas.
3. **Capa de Datos:** SQLAlchemy gestiona los modelos (`Manga`, `Chapter`, `User`).
4. **Seguridad:** Dependencias de FastAPI inyectadas para verificar tokens OAuth2 JWT.
5. **Cache/CDN:** Resoluciones de URL de imágenes apuntando directamente al CDN B2 respaldado por Cloudflare Workers.

1.4. Flujo de Datos

1. El cliente solicita un token JWT vía `/api/auth/token`. 2. Las peticiones subsiguientes incluyen el header `Authorization: Bearer <token>`. 3. Los Middlewares (ej. CORS, Throttling) y Dependencias interceptan la petición. 4. El servicio correspondiente (ej. `MangaService`) recupera datos de la base de datos usando SQLAlchemy. 5. Pydantic serializa la respuesta, optimizando y validando los datos salientes de forma síncrona/asíncrona.

2. Base de Datos y Modelos

2.1. Esquema Relacional

El sistema utiliza un esquema relacional gestionado mediante SQLAlchemy y control de versiones de esquema con Alembic.

2.1.1. Modelo de Usuarios (User)

- **User:** Modelo centralizado con email como identificador principal. Almacena:
 - `is_active`, `is_superuser`: Permisos base.
 - `role`: Rol funcional (reader, moderator, admin).
 - `is_nsfw_allowed`: Booleano para control parental/contenido sensible.

2.1.2. Modelo de Mangas (Manga)

La entidad central **Manga** se relaciona con:

- **Estado:** FK a la tabla **Estado** (En emisión, Finalizado).
- **Demografía:** FK a **Demografia** (Seinen, Shonen).
- **Géneros/Tags:** M2M mediante tabla intermedia para categorización.
- **Cover:** Relación para almacenar la portada principal y portadas secundarias.

2.1.3. Modelo de Estructura

Tablas auxiliares para estandarización:

- **Demografia:** Clasificación de audiencia con soporte de color hexadecimal.
- **Estado:** Ciclo de vida del contenido.
- **Chapter:** Capítulos vinculados a cada Manga con arrays de páginas en JSON.

2.2. Políticas de Integridad

- **Integridad Referencial:** Restricciones y cascadas configuradas en SQLAlchemy.
- **Transacciones:** Se manejan mediante `SessionLocal` y operaciones de commit/rollback explícitas.

3. Protocolos de Seguridad

3.1. Control de Acceso (RBAC)

El sistema implementa un control de acceso basado en roles mediante Dependencias de FastAPI:

1. **Anónimo:** Lectura de catálogo general y lectura de capítulos.
2. **Registrado (`get_current_user`):** Acceso a marcadores y lectura restringida.
3. **Verificado (+18):** Acceso a NSFW filtrado a nivel de serialización/base de datos.
4. **Admin (`get_admin_user`):** Permisos de escritura y acceso a endpoints de administración.

3.2. Autenticación

Se utiliza el estándar ****OAuth2 con JWT (JSON Web Token)****.

- **Access Token:** Firmado criptográficamente (HS256). Vida útil configurable (ej. 30 minutos).
- **Almacenamiento:** El cliente (Vue) guarda el token en memoria o `localStorage` y lo anexa en el header `Authorization: Bearer`.

3.3. Protección de Infraestructura

3.3.1. Rate Limiting

Implementado mediante dependencias o middlewares de Throttling personalizados para FastAPI:

- Endpoints públicos: Límites más estrictos para frenar scraping.
- Rutas autenticadas: Límites extendidos según rol.

3.3.2. Protección de CDN (Hotlink Protection)

Los recursos estáticos pesados (imágenes almacenadas en Blackblaze B2) están protegidos mediante un Worker de Cloudflare en Edge. Éste verifica el header `Referer` y el `Origin`, bloqueando todo acceso directo no autorizado e incrustaciones desde dominios externos maliciosos.

3.4. CORS

Políticas estrictas gestionadas por el `CORSMiddleware` de FastAPI, permitiendo solicitudes provenientes únicamente de orígenes pre-aprobados (ej. el frontend en producción y `localhost` en desarrollo).

4. Manual de Operaciones y Despliegue

4.1. Ciclo de Vida del Software (SDLC)

El desarrollo sigue un flujo ágil e iterativo: 1. **Desarrollo Local:** API testada con `uvicorn` y frontend con `Vite`. 2. **Gestión de Paquetes:** Entorno manejado con `pip` / `uv` para el backend y `npm` para el frontend. 3. **Build Backend:** Empaquetado y despliegue automático en Railway u otro PaaS. 4. **Migraciones de DB:** Se aplican los cambios de esquema mediante `alembic upgrade head`. 5. **Protección CDN:** Despliegue de reglas en Cloudflare Workers usando `wrangler deploy`.

4.2. Monitorización y Alertas

- **Disponibilidad:** Endpoint `/api/health` revisa el estado de conexión a la base de datos MySQL.
- **Errores API:** Registro directo mediante las excepciones de FastAPI.
- **Logs:** Logs asíncronos de `uvicorn` accesibles directamente desde el panel del proveedor cloud.

4.3. Procedimientos de Recuperación (Disaster Recovery)

4.3.1. Restauración de Servicio

En caso de fallo crítico en el despliegue del backend: 1. Acceder al Dashboard del PaaS (ej. Railway). 2. Seleccionar el commit estable anterior y ejecutar un "Rollback". 3. El contenedor revertirá su imagen en pocos segundos.

4.3.2. Backup de Datos

La base de datos MySQL cuenta con backups gestionados automáticamente por el proveedor de alojamiento.

5. Referencia API

Esta sección detalla los endpoints operativos para integración.

5.1. Autenticación

```
1 POST /api/auth/token
```

Body:

```
1 {  
2   "username": "usuario",  
3   "password": "password"  
4 }
```

5.2. Mangas

```
1 GET /api/manga/mangas/?limit=20  
2 GET /api/manga/mangas/{id}/
```

5.3. Monitorización

```
1 GET /api/health/
```